

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

[Наименование образовательной организации]

[Факультет] / [Кафедра]

ОТЧЁТ

по дисциплине «Проектная деятельность»

**Тема: «Итеративный состязательный атакователь ML-модели
с защитой через состязательное обучение»**

Выполнил: студент 3 курса
направление «Прикладная математика и информатика»

[ФИО студента] _____

Проверил: [ФИО руководителя] _____

[Город], 2026

СОДЕРЖАНИЕ

Введение.....	3
1 Цель и задачи	3
2 Теоретическая часть.....	4
2.1 Состязательные примеры	4
2.2 Атака FGSM.....	4
2.3 Атака PGD.....	4
2.4 Adversarial Robustness Toolbox (ART).....	5
2.5 Состязательное обучение	5
3 Данные и модель	5
3.1 Набор данных MNIST.....	5
3.2 Архитектура модели	5
4 Методика эксперимента	6
4.1 Итеративный цикл «атака — защита»	6
4.2 Форматы модели: pickle и ONNX.....	6
4.3 Параметры эксперимента	6
5 Программная реализация.....	7
6 Результаты.....	7
6.1 Эффект атаки на базовую модель.....	7
6.2 Динамика по итерациям	7
6.3 Успех атаки	9
6.4 Примеры состязательных изображений	9
6.5 Устойчивость при разной силе атаки.....	10
7 Обсуждение.....	11
Выводы	11
Список литературы	12

Введение

Современные модели машинного обучения, в особенности глубокие нейронные сети, достигли высокого качества в задачах классификации изображений. Однако в 2013–2014 годах было обнаружено, что такие модели уязвимы к состязательным примерам (adversarial examples) — намеренно искажённым входным данным, визуально неотличимым от исходных, но приводящим модель к ошибочным предсказаниям с высокой уверенностью.

Эта уязвимость представляет угрозу безопасности систем, использующих ML: от систем распознавания дорожных знаков до медицинской диагностики и биометрии. Поэтому изучение атак и методов защиты — актуальная задача доверенного искусственного интеллекта (Trustworthy AI).

В настоящей работе разработана программная система, которая в автоматическом итеративном режиме демонстрирует противоборство «атака ↔ защита»: модель атакуется состязательной атакой из библиотеки Adversarial Robustness Toolbox (ART), её качество под атакой падает, после чего применяется защита — состязательное обучение (adversarial training), — и качество восстанавливается. Цикл повторяется 10 итераций.

1 Цель и задачи

Цель работы — разработать итеративный «атакователь» ML-модели и продемонстрировать эффективность состязательного обучения как защиты, проведя 10 итераций цикла «атака — защита» с измерением метрик качества.

Для достижения цели поставлены следующие задачи:

- обучить базовую модель-классификатор (свёрточную нейросеть) на наборе данных MNIST;
- реализовать модуль-«атакователь», принимающий модель из файла и применяющий атаки из ART (FGSM и PGD);
- реализовать модуль защиты на основе состязательного обучения;

- реализовать оркестратор итеративного цикла на 10 итераций с логированием метрик;
- визуализировать результаты и проанализировать динамику робастности;
- экспортировать итоговую устойчивую модель в формат ONNX для переносимости.

2 Теоретическая часть

2.1 Состязательные примеры

Состязательный пример — это вход $x' = x + \delta$, где возмущение δ ограничено по норме ($\|\delta\|_\infty \leq \epsilon$) и визуально незаметно, но модель классифицирует x' неверно. Существование таких примеров впервые показано в работе Szegedy и соавторов (2013), а гипотеза линейности, объясняющая их, предложена Goodfellow и соавторами (2014).

2.2 Атака FGSM

FGSM (Fast Gradient Sign Method) — одношаговая атака, сдвигающая вход на величину ϵ в направлении знака градиента функции потерь по входу:

$$x' = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)).$$

Атака быстрая, но относительно слабая, так как использует лишь один шаг.

2.3 Атака PGD

PGD (Projected Gradient Descent) — итеративное усиление FGSM: на каждом шаге делается малый шаг по знаку градиента с последующей проекцией на ϵ -окрестность S исходной точки:

$$x^{t+1} = \text{Proj}_S(x^t + \alpha \cdot \text{sign}(\nabla_x J(\theta, x^t, y))).$$

PGD считается «сильнейшей атакой первого порядка» и используется как стандарт для оценки устойчивости. В работе PGD применяется как основная атака для оценки и как внутренний максимизатор при состязательном обучении.

2.4 Adversarial Robustness Toolbox (ART)

ART — открытая библиотека (проект Linux Foundation AI) для исследования устойчивости ML. Она предоставляет единый интерфейс «классификатор + атака»: модель оборачивается в `PyTorchClassifier`, после чего к ней применяются атаки `FastGradientMethod`, `ProjectedGradientDescent` и другие. В проекте ART используется как источник атак и как обёртка модели при оценке.

2.5 Состязательное обучение

Состязательное обучение (adversarial training, Madry и соавторы, 2017) — защита, при которой модель дообучается на состязательных примерах. Формально это решение минимаксной задачи:

$$\min_{\theta} E_{(x,y)} [\max_{\{\|\delta\|_{\infty} \leq \varepsilon\}} J(\theta, x + \delta, y)].$$

Внутренний максимум приближается атакой (PGD), внешний минимум — обычным шагом оптимизации. Ключевая деталь корректной реализации (см. раздел 4): состязательные примеры должны генерироваться заново для каждого батча против текущих весов модели.

3 Данные и модель

3.1 Набор данных MNIST

Используется классический набор MNIST: 70 000 изображений рукописных цифр размером 28×28 пикселей в градациях серого (60 000 — обучение, 10 000 — тест), значения нормированы в диапазон [0, 1]. Загрузка выполняется средствами ART (`art.utils.load_mnist`) с автоматическим кэшированием.

3.2 Архитектура модели

Модель — свёрточная нейронная сеть (CNN): два свёрточных слоя 3×3 (32 и 64 канала) с функцией активации ReLU, слой подвыборки `MaxPool 2×2`, прореживание (Dropout), затем полносвязный слой на 128 нейронов и выходной слой на 10 классов. Базовая точность на чистом тесте составила 98.8 %.

4 Методика эксперимента

4.1 Итеративный цикл «атака — защита»

Базовая модель сохраняется в файл и подаётся «на вход» атакующему (этим реализовано требование «на вход приходит ML-модель в виде файла»).

Далее выполняется 10 итераций, на каждой из которых:

- измеряется точность под атакой PGD (и FGSM) — это качество ДО защиты текущей итерации (оно падает);
- выполняется одна эпоха состязательного обучения — защита;
- повторно измеряется точность под PGD — качество ПОСЛЕ защиты (растёт);
- метрики логируются, модель итерации сохраняется в файл.

Оценка устойчивости на каждой итерации ведётся атакой PGD из ART при фиксированной конфигурации ($\epsilon = 0.2$, 40 шагов), поэтому метрики сопоставимы между итерациями, а кривая робастности отражает реальный прогресс.

4.2 Форматы модели: pickle и ONNX

Канонический формат пайплайна — PyTorch .pt (сериализация pickle): атакующий принимает модель именно как файл .pt. Итоговая устойчивая модель дополнительно экспортируется в формат ONNX (torch.onnx.export) и проверяется через onnxruntime; максимальное расхождение выходов PyTorch и ONNX составило порядка $1 \cdot 10^{-5}$, что подтверждает корректность экспорта и переносимость модели.

4.3 Параметры эксперимента

Параметр	Значение
Набор данных	MNIST, значения в $[0, 1]$
Базовое обучение	Adam, lr = $1e-3$, batch = 128, 3 эпохи
Атака PGD (оценка)	L_∞ , $\epsilon = 0.2$, шаг = 0.01, 40 итераций
Атака PGD (обучение защиты)	L_∞ , $\epsilon = 0.2$, шаг = 0.05, 7 итераций, генерация per-batch
Атака FGSM (сравнение)	L_∞ , $\epsilon = 0.2$
Число итераций цикла	10 (по 1 эпохе обучения)
Обучающая подвыборка	20 000 изображений
Подвыборка для оценки атак	2 000 тест-изображений

Параметр	Значение
Устройство	GPU NVIDIA (CUDA)

5 Программная реализация

Проект организован модульно; каждый модуль имеет одну зону ответственности:

- data.py — загрузка MNIST; model.py — архитектура CNN и сохранение/загрузка .pt;
- attacker.py — обёртка модели в ART и атаки FGSM/PGD;
- defense.py — состязательное обучение; pipeline.py — оркестратор 10 итераций;
- plots.py — графики; export_onnx.py — экспорт в ONNX.

Запуск всего эксперимента выполняется одной командой (python run_all.py); для разворачивания на новой машине предусмотрен установщик install.py, создающий изолированное окружение. Ниже приведён ключевой фрагмент модуля защиты — генерация PGD-примеров для каждого батча против текущих весов:

```
def adversarial_train_epoch(model, optimizer, x, y, device,
                           eps=0.2, alpha=0.05, steps=7, batch_size=128):
    model.train()
    perm = torch.randperm(x.shape[0], device=device)
    for i in range(0, x.shape[0], batch_size):
        b = perm[i:i + batch_size]
        xb, yb = x[b], y[b]
        x_adv = pgd_perturb(model, xb, yb, eps, alpha, steps) # против текущих весов
        optimizer.zero_grad()
        loss = F.cross_entropy(model(x_adv), yb)
        loss.backward()
        optimizer.step()
```

6 Результаты

6.1 Эффект атаки на базовую модель

Базовая модель показывает на чистом тесте точность 98.8 %. Однако под атакой её качество катастрофически падает: PGD при $\epsilon = 0.2$ снижает точность до 2.0 % (успех атаки 98 %), а более слабая атака FGSM — до 48.9 %. Это подтверждает уязвимость обычной модели и то, что PGD существенно сильнее FGSM.

6.2 Динамика по итерациям

В таблице 1 приведены метрики по 10 итерациям цикла (в процентах). Видно, что после первой же итерации состязательного обучения точность под PGD выросла с 2.0 % до 80.5 %, а к десятой итерации устойчивость достигла 92.3 % при сохранении точности на чистом тесте около 98.7 %. Успех атаки снизился с 98 % до 8.4 %.

Таблица 1 — Метрики по итерациям, %

Итер.	Чистый тест	FGSM до	PGD до	PGD после	Успех атаки
1	98.8	48.9	2.0	80.5	98.0
2	98.5	88.1	80.5	85.8	19.6
3	98.5	91.1	85.9	87.2	14.2
4	98.4	92.3	87.2	89.4	12.9
5	98.7	93.4	89.2	89.4	10.9
6	98.7	93.4	89.4	90.5	10.6
7	98.8	93.5	90.3	90.8	9.8
8	98.7	94.7	90.7	91.6	9.4
9	98.7	95.3	91.5	91.8	8.5
10	98.7	95.0	91.7	92.3	8.4

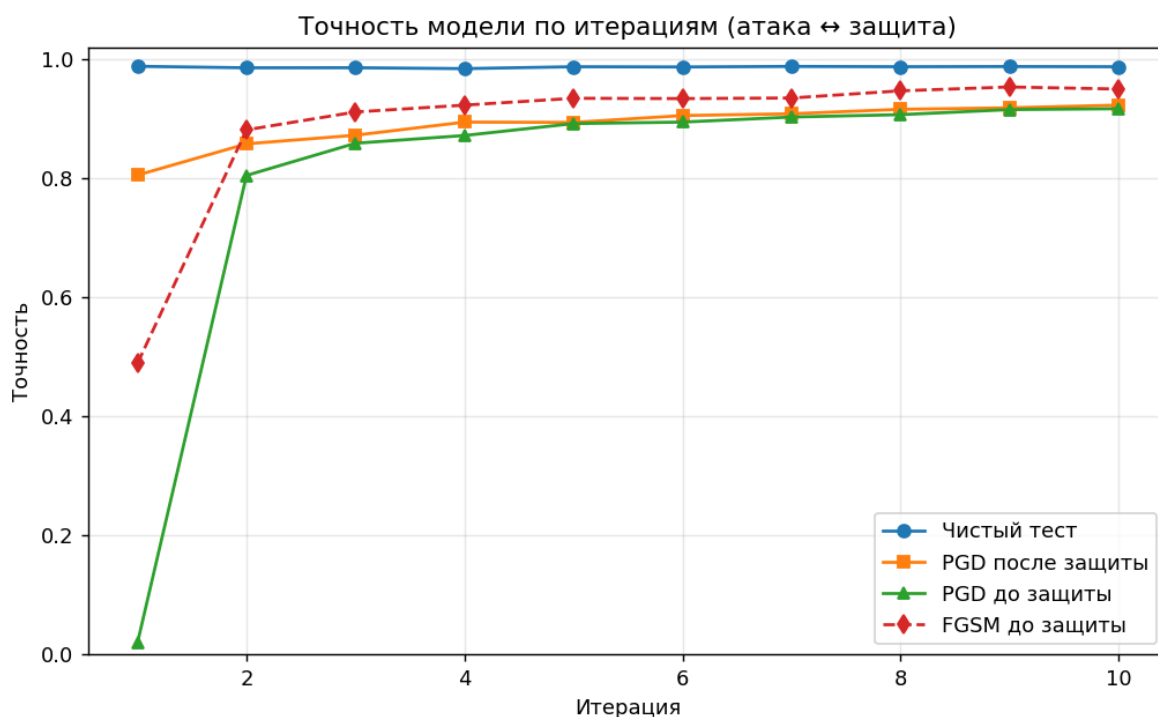


Рисунок 1 — Точность модели по итерациям (атака ↔ защита)

На рисунке 1 чистая точность (синяя линия) стабильна, а точность под PGD до защиты (зелёная) растёт от 2 % до 92 %, отражая накопление

устойчивости. Кривая FGSM лежит выше PGD, что согласуется с тем, что FGSM — более слабая атака.

6.3 Успех атаки



Рисунок 2 — Доля успешных атак PGD по итерациям

Рисунок 2 показывает падение доли успешных атак: если на базовой модели PGD успешен в 98 % случаев, то после состязательного обучения — лишь в 8 %.

6.4 Примеры состязательных изображений

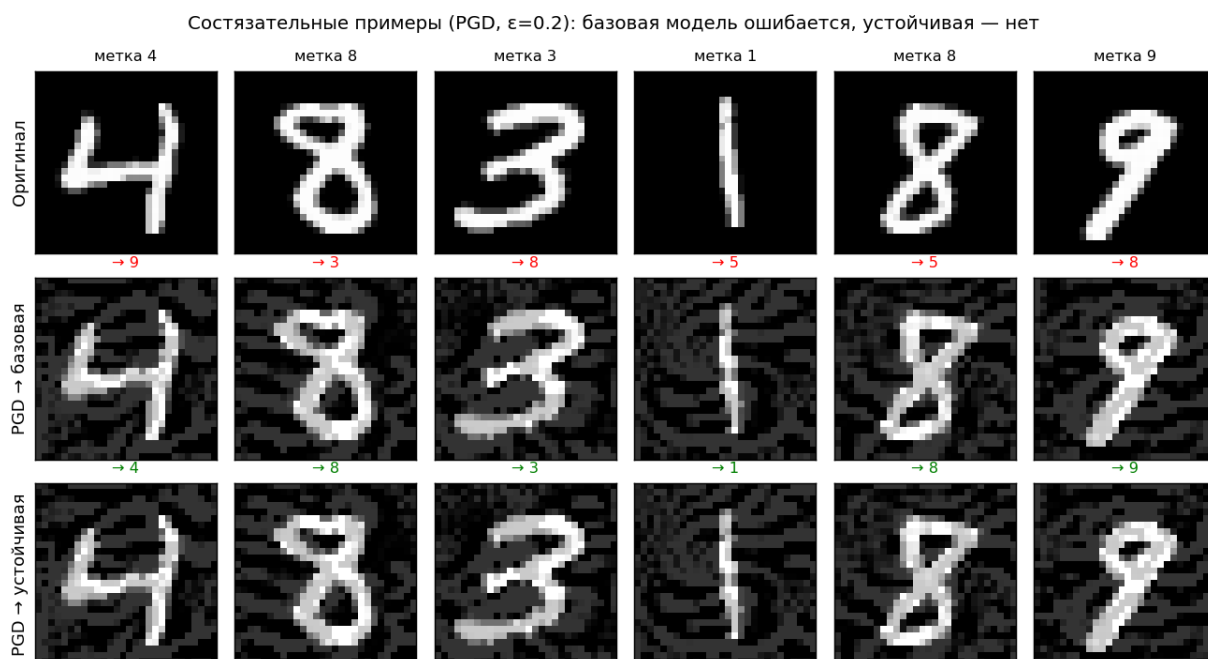


Рисунок 3 — Оригинал и состязательный пример (предсказания базовой и устойчивой моделей)

На рисунке 3 видно, что одно и то же PGD-возмущение обманывает базовую модель (предсказания во втором ряду неверны, отмечены красным), но не итоговую устойчивую модель (третий ряд, верные предсказания отмечены зелёным). Возмущение визуально проявляется как слабый «шум» вокруг цифры.

6.5 Устойчивость при разной силе атаки

В дополнительном эксперименте сравнивалась устойчивость базовой и итоговой моделей при разной силе атаки ϵ (таблица 2, рисунок 4). Устойчивая модель превосходит базовую при всех ϵ ; при $\epsilon = 0.2$ (на которое велось обучение) её точность 92.4 % против 2.2 % у базовой. При $\epsilon > 0.2$ устойчивость закономерно снижается, так как обучение велось на $\epsilon = 0.2$.

Таблица 2 — Точность под PGD при разной силе атаки, %

ϵ (сила атаки)	Базовая модель	Устойчивая модель
0.05	96.4	98.5
0.10	79.9	97.5
0.15	26.6	95.2
0.20	2.2	92.4
0.25	0.7	82.9
0.30	0.7	42.2

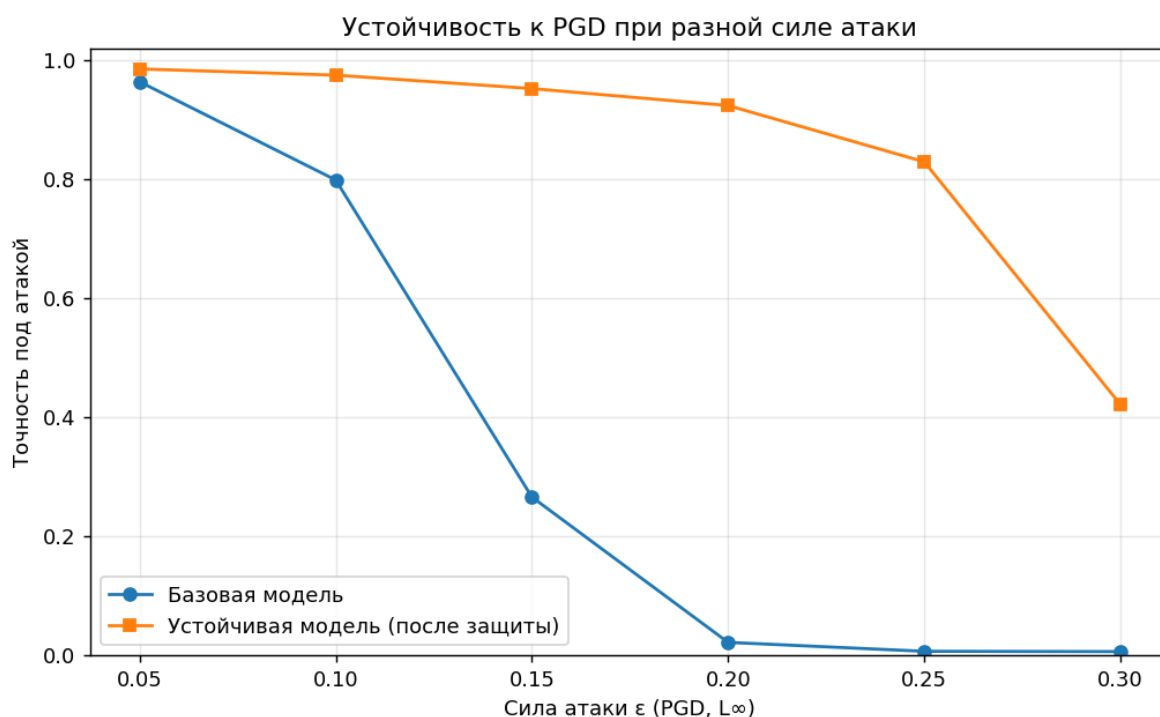


Рисунок 4 — Устойчивость к PGD при разной силе атаки ϵ

7 Обсуждение

В ходе работы был выявлен и устранён характерный методический подвох. В первой версии защиты состязательные примеры генерировались один раз за итерацию по фиксированной подвыборке: при этом точность под адаптивной атакой не росла (оставалась около 1–12 %), хотя на вид «обучение шло». Причина в том, что модель «запоминала» конкретные возмущения, но оставалась уязвимой к свежей атаке, перестроенной под обновлённые веса.

Решение — генерировать состязательные примеры заново для каждого батча против текущих весов модели (корректная реализация подхода Madry). После этой правки устойчивость к PGD стала уверенно расти: 2.0 % $\rightarrow$ 80.5 % $\rightarrow$... $\rightarrow$ 92.3 %. Этот результат подчёркивает, что качество защиты определяется не фактом обучения «на adversarial-данных», а тем, насколько атака при обучении адаптивна к модели.

Выводы

В работе разработан итеративный «атакователь» ML-модели и реализована защита через состязательное обучение. Получены следующие результаты:

- атака PGD из ART снижает точность базовой модели с 98.8 % до 2.0 % (успех атаки 98 %);
- состязательное обучение за 10 итераций повышает устойчивость к PGD до 92.3 % при сохранении точности на чистых данных (~98.7 %);
- успех атаки падает с 98 % до 8 %; устойчивая модель превосходит базовую при всех значениях ϵ ;
- итоговая модель экспортирована в ONNX (расхождение с PyTorch $\sim 1 \cdot 10^{-5}$);
- показано, что ключ к работающей защите — адаптивная (per-batch) генерация состязательных примеров.

Таким образом, цель работы достигнута: продемонстрировано противоборство «атака — защита» и эффективность состязательного обучения как меры повышения робастности.

Список литературы

1. Szegedy C. et al. Intriguing properties of neural networks. — arXiv:1312.6199, 2013.
2. Goodfellow I., Shlens J., Szegedy C. Explaining and Harnessing Adversarial Examples. — ICLR, 2015.
3. Madry A. et al. Towards Deep Learning Models Resistant to Adversarial Attacks. — ICLR, 2018.
4. Nicolae M.-I. et al. Adversarial Robustness Toolbox v1.0.0. — arXiv:1807.01069, 2018.
5. Adversarial Robustness Toolbox — документация. URL: <https://adversarial-robustness-toolbox.readthedocs.io>
6. LeCun Y., Cortes C., Burges C. The MNIST database of handwritten digits.